

AV2 — Teoria da Computabilidade

λ -Cálculo · MT Não Determinística

Turma CC5NA — Prof. Daniel Leal Souza · Semestre 01/2026

Equipe: Yuri Aguiar · João Rath · Pedro Paulo

Modelos escolhidos (Seção 3):

- Opção 7 — λ -Cálculo $\rightarrow$ *computar* o **fatorial** via combinador Y
- Opção 4 — MT Não Determinística $\rightarrow$ *decidir* $L = \{ ww : w \in \{a,b\}^+ \}$

github.com/yuriaguiardev/teoria-computabilidade

λ -Cálculo: sintaxe e a única regra

Termo (definição indutiva):

```
M ::= x | (λx. M) | (M N)
      variável | abstração | aplicação
```

- Variáveis livres × ligadas; α -equivalência ($\lambda x.x \equiv \lambda y.y$).
- Única regra — β -redução: $(\lambda x. M) N \rightarrow M[x := N]$.
- Substituição com captura evitada: ao entrar em $\lambda y.P$ com $y \in FV(N)$, renomeia-se y (α -conversão) para um nome fresco.

Tudo é função; a computação é só aplicar e substituir. É o coração do avaliador em `lambda_calc.py`.

Codificações de Church e ordem normal

- Numeral de Church: $n = \lambda f\ x. f\ (f\ \dots\ (f\ x))$ (aplica `f` `n` vezes).
- Booleanos: `TRUE` = $\lambda x\ y. x$, `FALSE` = $\lambda x\ y. y$, `IF` `b` `t` `e` = `b` `t` `e`.
- Aritmética: `MULT` = $\lambda m\ n\ f. m\ (n\ f)$; `ISZERO`, `PRED` (via pares).
- Estratégia — ordem normal (leftmost-outermost): pelo Teorema da Padronização, acha a forma normal se ela existir.

Não usamos o `int` do Python: o número 3 é a função que aplica algo 3 vezes (`prelude.py`). Ordem normal é o que faz `IF` e `Y` terminarem.

Recursão sem recursão: combinador Y e o problema

- O λ -cálculo puro não tem recursão nativa.

- **Combinador de ponto fixo Y:**

$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$, com $Y g = g (Y g)$.

- **Problema-alvo:**

```
FACT = Y (\f n. IF (ISZERO n) 1 (MULT n (f (PRED n))))
```

Y "amarra o nó": entrega a f uma cópia dela mesma, e assim o fatorial pode chamar a si próprio em $f (PRED n)$ — sem nenhuma palavra-chave de recursão.

Demonstração: `MULT 2 3` $\rightarrow$ `6` (traço completo)

```
python implementacoes/lambda_calculo/exemplos.py
```

7 β -reduções, do termo inicial à forma normal:

```
[0] (( $\lambda m\ n\ f.$  m (n f)) <2> <3>)          (2 e 3 expandidos em numerais de Church)
...
[7] ( $\lambda f\ x.$  f (f (f (f (f (f x))))))    = numeral 6
```

- `church_to_int` lê essa forma e decodifica $\rightarrow$ `6`.
- A resposta vem de **reescrita de termos**, não de aritmética da linguagem.

Custo real e o limite: FACT e a parada indecidível

Fatorial via Y — valores e custo medido em β -reduções:

Entrada	Resultado	β -reduções
FACT 2	2	248
FACT 3	6	1 525
FACT 4	24	10 384

$\Omega = (\lambda x. x x)(\lambda x. x x) \rightarrow \Omega \rightarrow \Omega \rightarrow \dots$ (NUNCA para)

- Nem todo termo tem forma normal $\rightarrow$ usamos **limite de passos**.
- Reflete a **indecidibilidade do problema da parada**
(testes `test_omega_nao_normaliza`, `test_captura_evitada`).

MTND: formalismo, relação δ e aceitação por ramo

7-upla $M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$ — δ é uma relação:

$$\delta : (Q \times \Gamma) \rightarrow P(Q \times \Gamma \times \{L, R, S\})$$

- Um mesmo (estado, símbolo) $\rightarrow$ **vários destinos** $\Rightarrow$ a computação é uma **árvore** de configurações (não um caminho).
- **Aceitação**: existe **algum** ramo que atinge q_{Accept} .
- **Rejeição**: **todos** os ramos morrem (sem estado de rejeição dedicado).

Problema $L = \{ww\}$ e a estratégia de fita

$L = \{ ww : w \in \{a,b\}^+ \}$ — palavra concatenada consigo mesma. Não é palíndromo.

- $\Gamma = \{ a, b, A, B, Sa, Sb, C, _ \}$ — marcadores:
 - A / B : 1ª metade já casada · Sa / Sb : início (adivinhado) da 2ª metade ·
 c : 2ª metade já consumida.
- Não determinismo: em $qScan$, a máquina adivinha o ponto médio.
- Verificação determinística: casa 1ª e 2ª metades posição a posição, na mesma ordem; aceita se ambas esgotam juntas.

$\{ww\}$ não é livre de contexto → um autômato de pilha não basta; exige MT.

Tabela de transições e o ponto de não determinismo

- $|Q| = 10 (> 8) \cdot 37$ transições (≥ 10) — atende a Seção 7.
- Único ponto de não determinismo = estado `qScan` :

$\delta(qScan, a) = \{ \underset{\substack{\text{L seguir na 1ª metade}}}{(qScan, a, R)} , \underset{\substack{\text{L "aqui começa a 2ª metade"}}}{(qFindLeft, Sa, L)} \}$

- Pares `(estado, símbolo)` ausentes $\Rightarrow$ ramo morre.

Estado	lê <code>a</code>	lê <code>b</code>	lê <code>c</code>	lê <code>_</code>
<code>q0</code>	<code>qScan,a,R</code>	<code>qScan,b,R</code>	—	—
<code>qScan</code>	<code>qScan,a,R </code> <code>qFindLeft,Sa,L</code>	<code>qScan,b,R </code> <code>qFindLeft,Sb,L</code>	—	—
<code>qScanRight</code>	<code>qToBoundaryA,A,R</code>	<code>qToBoundaryB,B,R</code>	<code>qCheckSecond,C,R</code>	—

Demonstração: árvore, ramo aceitante e oráculo

```
python implementacoes/mt_nao_deterministica/maquina_ww.py
```

`ab` → **rejeita** (todos os ramos morrem):

```
q0: [a] b
├─ qScan: a [b]
│   └─ qScan: a b [_]      X (não tagueou → morre)
│   └─ qFindLeft: [a] Sb
│       └─ ... qToBoundaryA: A [Sb] X (a ≠ b → morre)
```

- `abab` → **ACEITA**: imprime o **ramo aceitante** (`q0 ... qAccept`).
- **Oráculo**: comparado com a definição de L em **todas as 2047** cadeias de `{a,b}*` com `|w| ≤ 10` → **0 divergências** (teste `test_oraculo`).

Análise: computabilidade e limites

- **Church-Turing:** λ -cálculo $\equiv$ MT $\equiv$ funções recursivas (Kleene, 1936). Tudo "efetivamente computável" cai numa dessas formulações.
- **Não determinismo é conveniência, não poder extra:** toda MTND é simulável por MT determinística (custo possivelmente exponencial).
- **Hierarquia de Chomsky:** $\{ww\}$ não é livre de contexto $\Rightarrow$ exige uma MT; um autômato de pilha não basta.
- **Limites:** parada indecidível — Ω no λ -cálculo; ramos não-parantes no caso geral $\Rightarrow$ ambos os simuladores usam **limite de passos**.

Conclusão

- **Entregamos os dois modelos** com implementação própria, testes e rastreamento:
 - λ funcional (reescrita real de termos, **fatorial via Y**);
 - MTND que **decide** `{ww}` (árvore, ramo aceitante, **oráculo**).
- **Reprodutível:** Python 3.10+, **só biblioteca padrão; 32 testes**
(`python -m unittest discover -s testes`).
- **Formalismo explícito:** parser + β -redução; δ como relação + árvore — nada de "função pronta" devolvendo a resposta.

Referências: Sipser (2013); Pierce (2002); Barendregt (1984); Diverio & Menezes (2011); Church (1936); Turing (1936).

Equipe: Yuri Aguiar · João Rath · Pedro Paulo — CC5NA · Perguntas?